붙임 3

연구논문/작품 최종보고서

2026 학년도 제 2 학기

제목 : LolAi.gg 딥러닝 기반
과거 경기 분석 및 승패 예측 시스템

박승현 (2019310202)

2026 년 10 월 30 일

지도교수: 한환수 서명

계획(10)	주제(20)	개념(20)	상세(30)	보고서(20)	총점(100)

* 지도교수가 평가결과 기재

■ 목차

■ 요약 1

■ 서론 2

■ 관련연구 8

챔피언 조합과 신경망 8

초기 정보와 변수 분석 8

일반 사용자와 분석 단위 8

다층 표현과 과적합 9

경기 시계열과 고정 스냅샷 9

본 작품의 위치 9

■ 제안 작품 소개 10

시스템의 목적과 전체 구성 10

요청 제어, 캐시와 자료 출처 11

경기 적격성 및 초기 특성 추출 12

신경망 구조와 학습·저장 14

분석 응답과 사용자 화면의 연결 16

■ 구현 및 결과분석 18

1. 구현 범위와 분석 흐름 18

2. 실제 데이터와 전처리 구현 18

3. 다층 신경망과 추론 구현 20

4. 평가 지표와 비교 조건 21

5. 측정 결과와 해석 22

6. 재현성·검증 결과와 남은 범위 24

7. 개인별 복기와 시간별 모형 확장 26

8. 자체 분석과 검증 확장 29

■ 결론 및 소감 33

결론 33

개인 소감 및 기여 확인 33

■ 참고문헌 35

추가 구현 참고자료 36

■ 부록 핵심 원천코드 37

backend/app/ml.py 37

backend/scripts/train_model.py 40

■ 요약

본 작품은 Riot API 의 완료된 리그 오브 레전드 경기를 직접 분석하는 LolAi.gg 이다. React·TypeScript 와 FastAPI, SQLite 를 연결하고 인증키는 서버에만 둔다. 여섯 팀 차이를 입력하는 6-16-8-4-1 다층 신경망과 11 개 시간별 모형을 서비스에 연결하였다. 최종 0.6.0 에는 10 인 전적표, 역할 상대 비교, 자체 챔피언·빌드 집계, 설명 가능한 복기 우선순위, 구매·되돌리기 및 스킬 순서, 날짜별 랭크 기록과 개인 연습 추적을 구현하였다. 외부 통계 서비스는 선택 보조 자료이며 자체 분석을 대신하지 않는다.

원래 104 경기의 학습 83·평가 21 분할과 모형을 보존하였다. 주 MLP 의 기존 평가 정확도는 47.62%, ROC-AUC 0.49091, Brier 0.372683 이었다. 이후 별도 수집 180 경기 스냅샷에서 사전 제외 목록과 모형 생성 시각 조건을 적용한 35 경기를 추가 점검하였다. 변경하지 않은 MLP 는 20/35 인 57.14%, AUC 0.67, Brier 0.319481 이었고 로지스틱 회귀는 28/35 인 80.00%, 0.873333, 0.168850 이었다. 서로 다른 표본이므로 정확도 차이를 모형 개선으로 해석하지 않는다. 전체 백엔드 검사 276 개·하위 검사 6 개와 프런트엔드 검사 41 개 및 배포 빌드가 통과하였다.

핵심어: 리그 오브 레전드, 딥러닝, 다층 퍼셉트론, 시간순 평가, 과적합, 경기 후 분석

■ 서론

리그 오브 레전드의 한 경기는 두 팀이 일정 시간 동안 자원과 목표물을 확보하고 상대 팀과 경쟁한 결과로 기록된다. 경기 종료 화면의 승패와 개인별 합계 통계는 결과를 빠르게 확인하는 데 유용하지만, 그 수치만으로 경기 도중의 상태를 모두 이해하기는 어렵다. 같은 승리 경기라도 초반부터 우세했던 경우와 불리한 상태에서 결과를 뒤집은 경우가 있을 수 있으며, 개인의 처치 수와 팀 전체의 자원 상황이 같은 방향을 가리키는 것도 아니다. 따라서 완료된 경기 기록을 조회하는 기능에 더하여 개인 통계, 팀의 시간별 변화, 일정 시점에서의 결과 추정을 함께 살펴볼 수 있는 분석 환경을 구성할 필요가 있다.

이러한 분석에서는 정보를 많이 보여 주는 것만큼 각 정보의 의미와 이용 시점을 구분하는 일이 중요하다. 경기 종료 후 계산한 KDA 나 최종 피해량은 해당 경기의 활동을 정리하는 자료이며, 초반 시점에 알 수 있었던 정보와는 범위가 다르다. 또한 특정 수치가 높거나 낮았다는 관찰은 그 수치가 승패를 직접 결정했다는 설명과 구별되어야 한다. 사용자가 경기 후 기록을 검토하는 과정에서는 수치의 출처, 계산 기준, 분석 대상 시점과 해석의 한계를 함께 확인할 수 있어야 한다. 본 프로젝트는 이 구분을 화면과 자료 처리 과정에 반영하는 것을 경기 분석 시스템의 기본 요구로 설정하였다.

승패 추정 모형을 결합할 때에도 모형이 실행된다는 사실과 유용한 예측을 한다는 주장은 서로 다르다. 신경망의 층을 늘리거나 복잡한 이름을 사용하는 것만으로 새로운 경기에서 더 정확한 결과가 보장되지는 않는다. 특히 수집한 경기 수가 제한된 경우에는 학습 자료에 잘 맞는 결과가 이후 자료에서도 유지되는지 확인해야 한다. 이에 본 작품은 딥러닝 모형을 실제로 학습하고 서비스에 연결하되, 단순한 모형 및 고정 확률 기준선과

동일한 조건에서 비교하는 절차를 포함한다. 분석 기능의 구현 성과와 예측 실험에서 확인한 성과를 분리하여 제시하는 것이 연구 목표의 한 부분이다.

사용자는 Riot ID 와 지역을 입력하여 완료 경기를 페이지별로 조회하고, 역할·챔피언 통계와 선택 경기의 기록을 확인한다. 각 응답의 요약은 해당 페이지 기준이다. 실제 조회는 별도 SQLite 에 중복 없이 저장되며 자체 챔피언 집계와 복기 참고 표본을 함께 갱신한다. 원래 연구 JSONL 과 학습 모형은 변경하지 않는다. 최종 통계에 근거한 복기, 초기 시점 예측과 현재 랭크·외부 아카이브는 구분하여 표시한다.

범위는 완료 경기의 회고적 분석과 개인 연습 기록이다. 구매 순서와 스킬 선택, 설명 가능한 통계 기반 조언을 구현했으나 최적 아이템·전략 또는 승률 향상 효과를 입증하지 않았다. 비공개 자기·동료 의견 노트는 인증된 동료 평가가 아니다. 현재 랭크를 과거 경기의 실제 티어로 간주하지 않으며 저티어 특화 성능도 입증하지 않았다. 음성·실시간 코칭과 통계 기반 실력 티어 추정은 이번 범위에서 제외한다.

핵심 원칙은 경기 후 통계, 시점별 예측 입력, 현재 계정 상태의 분리이다. 최종 골드나 피해량을 초반 예측에 섞으면 당시 이용할 수 없던 정보로 성능을 계산하게 된다. 전체 경기 골드 그래프와 개인 비교는 사후 자료를 사용하지만, 모형 입력은 각 관측 프레임까지로 제한한다. 현재 랭크·챔피언 숙련도도 과거 경기 당시의 값으로 소급하지 않는다. 최종 승패는 지도학습과 평가의 정답으로만 사용한다.

예측 입력은 파란 팀 합계에서 빨간 팀 합계를 뺀 골드, 경험치, CS, 처치 수, 포탑 수, 드래곤 수의 여섯 가지 차이로 구성하였다. 초기 스냅샷은 경기 시작 후 540 초 이상 600 초 이하 범위에서 가장 늦은 완전한 프레임을 선택하며, 그 프레임 이후에 발생한 이벤트는 제외한다. 이번 실제 자료에서 선택된 프레임은 약 540.09 초로, 약 9 분 시점에

해당한다. 다음 프레임이 600 초를 조금 넘기 때문에 이를 정확히 10 분 시점의 관측으로 표현하지 않는다. 이러한 기준은 같은 자료에서 특성을 다시 추출할 때 입력 시점을 일관되게 유지하고, 미래 정보가 유입되었는지 점검하기 위한 기준이 된다.

시스템은 React 와 TypeScript 로 구성한 사용자 화면, FastAPI 기반 서버, Riot API 를 통한 경기 자료 수집 및 SQLite 캐시로 구성된다. 서버는 Riot ID 를 계정 식별자에 대응시킨 뒤 랭크 경기 목록, 경기 상세와 타임라인을 조회한다. 지역 경로의 허용 범위, 요청 간 간격과 호출 제한 응답을 처리하여 외부 자료 이용 과정의 오류를 사용자에게 전달한다. 인증키는 서버의 로컬 환경 설정에서 관리하며 프런트엔드 코드나 브라우저에 제공하는 번들에 포함하지 않는다. 외부 API 호출과 화면 표시 사이에 서버를 두어 자료 변환, 캐시 사용과 추론을 일관된 경로에서 처리하도록 구성하였다.

화면은 승률·KDA·분당 CS·킬 관여율·시야와 피해량, 역할·챔피언별 표본 수와 결과를 제공한다. 선택 경기를 제외한 자기 기록과 비교하고, 전체 골드 흐름과 시간별 승패 추정을 별도로 살펴볼 수 있다. 입력 민감도와 관측 이벤트 시각은 검토 단서를 제공하지만 패배 원인이나 권장 행동을 확정하지 않는다. 분석 JSON 내보내기에는 프로필, 개인 비교와 시간별 추정도 포함한다. 별도 요청으로 조회하는 현재 계정 상태와 영상 결과를 같은 내보내기에 포함했다고 주장하지 않는다.

외부 API 에 의존하는 작품은 인증키의 유효 여부, 호출 제한과 네트워크 상태에 따라 시연 가능성이 달라질 수 있다. 본 작품은 실제 조회 경로와 함께 캐시 및 합성 데모 경로를 마련하였다. 자동 모드에서 이용 가능한 캐시를 사용하거나 합성 자료로 전환하는 경우에는 출처와 경고를 표시하며, 합성 데모는 DemoPlayer 라는 가상 플레이어의 경기로 제시한다. 이를 통해 외부 연결이 어려워도 경기 선택, 통계 표시와

그래프 등 분석 화면의 흐름을 시연할 수 있다. 다만 실제 자료용 모형으로 합성 경기의 확률을 출력하거나 합성 모형을 실제 경기에 적용하지 않도록 자료 출처를 검사하며, 데모 작동을 실제 플레이어 조회의 성공으로 취급하지 않는다.

구현 결과물은 화면 구성에 한정되지 않는다. 실제 경기와 타임라인을 수집하고 중복을 제거하는 수집기, 초기 특성 추출과 시간순 분할을 수행하는 학습 경로, 모형의 가중치와 정규화 정보를 저장하는 JSON 형식, 저장된 모형으로 추론하는 서버 경로를 포함한다. 추론 시에는 자료 출처와 입력·모형의 차원, 활성화 및 유한한 수치 여부를 확인한다. 또한 실행 가능한 직렬화 파일을 불러오는 대신 저장된 수치로 순전파를 수행하며, 분석 캐시를 모형 내용에 따라 구분하여 모형 변경 전 확률이 재사용되지 않도록 하였다. 실행 안내와 검증 기록을 함께 정리하여 화면, 자료 처리와 모형의 연결을 확인할 수 있게 하였다.

주 딥러닝 모형은 여섯 입력에서 16 개·8 개·4 개 뉴런의 세 ReLU 은닉층을 거쳐 하나의 시그모이드 출력으로 연결되며, 학습 매개변수는 289 개다. 파란 팀 승리 확률을 계산하고 빨간 팀에는 여사건을 제공한다. 같은 분할의 로지스틱 회귀와 학습 파란 팀 승리 비율을 쓰는 상수 기준선을 비교하였다. 이후 같은 분할에서 Random Forest 를 추가 비교하였으며, 시간별 신경망은 각 시점의 독립 모형으로 학습하였다. 복잡한 모형의 구현과 예측 우수성을 같은 의미로 사용하지 않는다.

실험에는 실제 Riot API 에서 수집한 KR 랭크 솔로/듀오 경기 104 개를 사용하였다. 경기 시작 시각 순서로 앞선 83 개를 학습하고 이후 21 개로 평가하였으며, 하나의 경기에서 파란 팀 관점의 표본 하나만 생성하였다. 학습과 평가 사이에 경기 ID 가 겹치지 않도록 하고, 모든 학습 경기가 첫 평가 경기 시작 전에 종료되었는지 확인하였다. 정규화의

평균과 표준편차는 학습 분할에서만 계산하며, 비교 모형은 같은 경기와 특성을 사용한다. 다만 수집이 한 플레이어 및 해당 경기에서 확인한 참가자 다섯 명으로 시작되었으므로, 서로 연결된 경기와 플레이어 중복에 따른 선택 편향이 존재한다. 이 자료를 전체 이용자 집단을 대표하는 표본으로 간주하지 않는다.

주 신경망은 학습 경기 83 개 중 79 개를 맞혀 95.18%의 학습 정확도를 보였으나, 평가 경기에서는 21 개 중 10 개로 47.62%를 기록하였다. 로지스틱 회귀는 57.14%였고, 신경망의 Brier 는 두 기준선보다 나빴다. 추가 Random Forest 는 14/21 을 맞혔지만 Brier 는 상수 기준선보다 컸다. 시간별 신경망도 대부분의 시점에서 단순 기준선보다 큰 확률 오차를 보였다. 따라서 딥러닝의 학습과 서비스 추론을 연결했음에도 예측 품질 개선을 입증하지 못한 소규모 실험으로 해석한다.

평가 21 경기는 앞선 로지스틱 회귀에서 이미 확인한 자료다. 주 MLP 의 구조와 설정을 평가 점수를 높이기 위해 탐색하지 않았지만, Random Forest 및 시간별 모형을 추가한 비교 역시 새로운 독립 시험이 아니다. Random Forest 의 고정 설정도 평가셋을 확인한 뒤 정했으므로 탐색적 결과로 한정한다. 다른 플레이어·서버·패치의 일반화나 사용자 승률 향상은 검증하지 않았다. 보다 강한 결론에는 학습 내부의 설정 결정과 이전에 보지 않은 평가 자료가 필요하다.

작품의 평가는 기능 검증과 일반화 평가를 구분하여 수행한다. 기능 검증은 자료 조회와 표시, 입력 시점 제한, 팀 방향, 자료 출처 검사, 저장된 가중치와 서비스 추론의 일치 등 구현이 정한 규칙대로 동작하는지 확인하는 과정이다. 이러한 확인은 잘못된 입력이나 연결 오류를 발견하는 데 필요하지만, 모형이 새로운 경기의 결과를 정확하게 추정한다는 증거를 대신하지 않는다. 일반화 평가는 학습에 사용하지 않은 경기에서

기준선과 같은 조건으로 비교하고, 자료 분리와 표본의 독립성까지 검토해야 한다. 본 보고서는 학습 및 오프라인 추론 경로를 구현한 성과와 탐색적 예측 실험의 부정적 결과를 함께 제시하여, 구현의 타당성과 연구 주장의 범위를 각각 판단할 수 있도록 한다.

이하 관련연구에서는 경기 승패 예측에 사용된 정보의 시점, 분석 단위와 신경망 접근을 검토하고 본 작품과의 차이를 정리한다. 제안 작품 소개에서는 사용자 흐름과 서버 구조, 자료 출처와 전처리, 다층 신경망의 학습 및 추론 방법을 설명한다. 구현 및 결과분석에서는 실제 구현 경로와 검증 근거, 동일 분할의 비교 지표와 자료·평가 절차의 한계를 제시한다. 결론 및 소감에서는 확인된 결과를 종합하고 후속 과제를 정리하되, 학생 개인의 기여와 경험은 실제 수행 내용에 근거하여 구분한다. 이를 통해 본 작품의 범위, 재현 가능한 구현 결과와 아직 입증하지 못한 부분을 일관된 기준으로 전달하고자 한다.

■ 관련연구

챔피언 조합과 신경망

김철기·이수원은 경기 시작 시점의 챔피언 조합을 양방향 LSTM 임베딩으로 표현하는 승패 예측을 연구하였다[1]. 이는 게임 자료에 신경망을 적용한 국문 선행사례다. 본 작품은 챔피언 조합이나 순환신경망 대신 초기 팀 상태의 여섯 차이를 입력하는 피드포워드 모형을 사용한다. 입력 시점과 정보가 다르므로 선행연구의 성능을 본 작품의 성능으로 대체하지 않는다.

초기 정보와 변수 분석

김용우·김영민은 초반 10 분 자료의 승패 모형과 변수중요도를 검토하였다[2]. 초기 정보를 제한하고 분석한다는 문제의식이 관련된다. 본 작품의 실제 관측은 엄격한 600 초 상한 안의 약 9 분 프레임이며, 작은 편의표본의 탐색적 비교다. 같은 검증 조건의 재현 연구 또는 특정 행동의 인과효과 분석으로 표현하지 않는다.

일반 사용자와 분석 단위

김예지·민정혜는 Riot API 의 일반 사용자 자료에서 개인 및 팀 단위 모형을 구분하여 비교하였다[3]. 이는 자료 수집과 분석 단위를 명시할 필요성을 보여 준다. 본 작품은 팀 차이를 사용하고 예측 입력의 초기 시점 제한을 별도로 적용한다. 자료와 평가 조건이 같지 않으므로 연구 간 정확도의 단순 우열을 주장하지 않는다.

다층 표현과 과적합

LeCun 등의 논의는 여러 처리층을 통한 표현 학습과 매개변수 학습을 딥러닝의 배경으로 설명한다[4]. 본 작품은 세 비선형 은닉층을 실제 학습하지만, 깊이가 존재한다는 사실만으로 작은 자료에서의 예측 우수성이 보장되지는 않는다.

Srivastava 등은 학습 중 유닛을 확률적으로 제외하는 dropout 을 과적합 대응 방법으로 제시하였다[5]. 본 작품에 적용된 것은 L2 정규화이며 dropout 은 구현하지 않았다. 이 문헌은 정규화 배경과 후속 비교 후보로 사용하고, 현재 적용된 기법처럼 기재하지 않는다.

경기 시계열과 고정 스냅샷

Silva 등은 프로 경기의 구간별 시계열에서 순환신경망 계열을 비교하였다[6]. 본 작품은 KR 랭크 경기의 시점별 독립 스냅샷을 사용하며, 프레임 간 순서 자체를 신경망의 기억 상태로 학습하지 않는다. 확률 곡선은 서로 다른 시점에 맞춰 학습한 MLP 를 과거 관측에 적용한 결과다. LSTM 이나 실시간 코칭으로 설명하지 않으며, 자료와 평가 절차가 다른 연구의 정확도를 직접 비교하지 않는다.

본 작품의 위치

선행연구를 비교할 때는 정보 이용 시점, 분석 단위, 표본, 전처리의 학습 범위와 평가 독립성을 확인해야 한다. 본 작품의 기여는 실제 자료 조회·분석·안전한 추론의 통합 구현과 동일 조건에서의 신경망 비교 실험이다. 원문 확인 범위는 관련연구 문서 두 건에 기록하였다.

■ 제안 작품 소개

시스템의 목적과 전체 구성

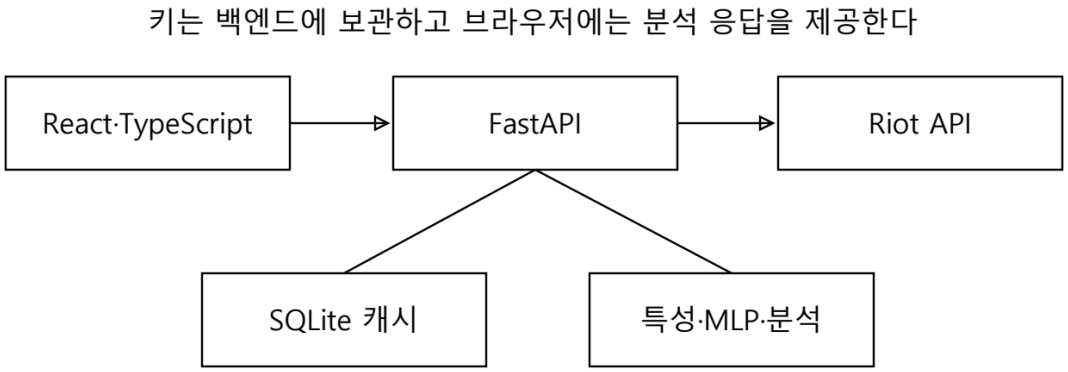


그림 1. 실제 구현의 자료 흐름과 백엔드 처리 경계

LolAi.gg 는 리그 오브 레전드의 완료된 랭크 경기를 조회하고, 플레이어의 경기 후 통계와 초기 팀 상태에 기반한 승패 추정치를 함께 제공하는 웹 시스템이다. 사용자는 Riot ID 와 지역, 자료 모드를 선택하여 최근 경기를 불러온다. 분석 대상은 소환사의 협곡 랭크 솔로/듀오 및 자유 랭크 경기이며, 실시간 경기 관측이나 게임 중 행동 지시는 구현하지 않았다. 화면에 표시하는 최종 통계와 전체 경기 골드 변화는 회고적 분석에 사용하고, 신경망 입력은 별도로 제한된 초기 관측에서 추출한다.

시스템은 React·TypeScript 화면, FastAPI 서버, Riot 연동, SQLite 캐시와 통계·모형 모듈로 구성된다. App.tsx 는 Riot ID 를 분리하여 /api/analysis 를 호출한다. 서버는 ACCOUNT-V1 에서 PUUID 를 얻고 MATCH-V5 의 랭크 경기 목록·상세·타임라인을 조회한다. analytics.py 는 경기 통계를, profiles.py 는 역할·챔피언 집계와 자기 기록 비교를 담당한다. ml.py 의 초기 추정과 temporal_model.py 의 시점별 추론을 화면에 함께 제공한다. 현재 랭크·숙련도와 영상은 별도 요청으로 조회한다.

외부 API 호출과 인증은 백엔드에 모았다. Settings 는 서버 환경에서 키를 읽고 Riot 요청에는 X-Riot-Token, YouTube 요청에는 X-Goog-Api-Key 헤더를 사용한다. 브라우저 요청이나 분석 응답에 키를 넣지 않는다. 계정·경기 영역과 현재 랭크·숙련도의 플랫폼 호스트는 허용목록에서만 선택하며, 외부 URL 을 사용자 입력으로 지정하지 않는다. 이름과 식별자는 경로에 맞게 인코딩한다. 현재 계정 상태는 과거 모형의 입력으로 사용하지 않는다.

요청 제어, 캐시와 자료 출처

RiotClient.get 은 캐시를 먼저 확인하고, 실제 네트워크 요청은 하나의 비동기 잠금 안에서 간격을 두어 수행한다. 기본 요청 간격은 1.25 초이며 시간 초과 기본값은 12 초이다. 잠금을 획득한 뒤 캐시를 다시 확인하여 같은 자료에 대한 중복 호출을 줄인다. 429 응답을 받으면 Retry-After 를 초 단위 값 또는 HTTP 날짜로 해석하고, 해석할 수 없는 경우 60 초를 적용한다. 계산한 대기 종료 시각은 클라이언트 전체에서 공유한다. 대기 중 들어온 호출은 즉시 재시도하지 않고 남은 시간을 포함한 오류를 반환하며, FastAPI 오류 처리기는 상태 코드와 Retry-After 헤더를 전달한다. 이 기능은 외부 제한에 대응하는 단일 실행 인스턴스의 제어이며 분산 요청 제어를 구현한 것은 아니다.

SQLite 캐시는 응답 JSON 과 생성·만료 시각을 저장한다. 계정·현재 랭크·숙련도는 최대 5 분, 경기 목록은 2 분, 완료 경기 상세·타임라인은 7 일 동안 재사용한다. 통합 분석 키에는 지역, 정규화한 Riot ID, 요청 경기 수, 응답·추론 버전과 주 모형·시간별 모형·비교 보고서의 내용 해시를 반영한다. 새 JSON 모형이나 평가 자료가 설치되면 이전 분석 결과와 구분하며, 외부 조회 후 저장하기 전에 버전을 다시 확인한다. 성공한 영상 결과는 최대 6 시간 캐시하고 실패 결과는 저장하지 않는다.

자료 모드는 demo, auto, live 의 세 가지이다. demo 는 DemoPlayer 라는 가상 플레이어와 합성 경기를 반환한다. auto 와 live 는 유효한 캐시가 있으면 이를 사용하고, 없으면 Riot 조회를 수행한다. live 에서 외부 조회가 실패하면 오류를 반환한다. auto 에서는 플레이어·경기를 찾을 수 없는 404 를 그대로 전달하며, 그 밖의 조회 오류에는 동일 모형 버전의 이전 분석을 최대 7 일까지 대체 자료로 사용할 수 있다. 이전 분석도 없으면 합성 데모를 반환하되, 응답의 플레이어는 가상 신원을 유지하고 requested_player 에 사용자가 요청한 신원을 별도로 기록한다. 실제 플레이어의 결과처럼 합성 기록을 표시하지 않도록 한 것이다.

응답의 source 는 새 Riot 호출을 포함한 live, 저장된 Riot 자료인 cache, 합성 자료인 demo 로 구분한다. 캐시를 거쳤다는 사실은 원자료가 합성이라는 뜻이 아니므로, 개별 Riot 경기의 추론 출처는 riot 로 유지한다. 일부 경기 상세가 사라졌거나 타임라인만 누락된 경우에는 가능한 기록을 유지하고 경고를 제공한다. 타임라인이 없는 경기는 최종 통계를 표시할 수 있어도 초기 예측은 제공하지 않는다. 모형의 trained_on 과 경기의 riot 또는 synthetic 출처가 다르면 확률 계산을 거부한다. 출처 표시는 프로그램이 관리하는 메타데이터이므로, 연구 자료의 진위 확인 자체를 대체하지는 않는다.

경기 적격성 및 초기 특성 추출

화면 분석 전의 eligible_match 는 맵 11, 큐 420 또는 440, 600 초를 초과한 경기 길이, 열 명의 참가자와 팀별 다섯 명 구성을 확인한다. 경기 식별자와 시작 시각, 참가자 식별자, 승패 형식 및 필요한 최종 통계의 유한 수치도 검사한다. 조기 종료 표시가 있거나 기록이 불완전한 경기는 제외한다. 이 검사는 최종 통계를 안전하게 계산하기

위한 절차이다. 예측용 extract_features 는 그와 별개로 초기 프레임과 이벤트의 적격성을 확인하므로, 통계가 표시되는 모든 경기에 예측값이 존재하는 것은 아니다.

특성 추출에서는 경기 상세와 타임라인의 matchId 가 같아야 한다. 팀 대응은 참가자 메타데이터에서 만들고 번호만으로 추정하지 않는다. 주 모형은 540,000~600,000ms 의 가장 늦은 완전한 프레임을 사용한다. 시간별 모형은 각 목표 시각 직전 1 분 구간에 같은 규칙을 적용한다. 열 명의 관측과 골드·경험치·CS 의 음수가 아닌 유한 수를 확인하고, 구간 밖의 과거 프레임이나 상한을 넘긴 미래 프레임으로 대신하지 않는다.

기존 실제 자료 104 경기에 선택된 프레임 시각은 540.088~540.308 초이다. 다음 프레임이 600 초를 조금 넘기 때문에, 본 작품에서 사용하는 관측은 실제로 약 9 분 시점이다. 코드와 화면의 minute-10 명칭은 10 분까지 이용 가능한 정보라는 상한을 나타내며 정확히 600 초에 측정했다는 의미로 해석하지 않는다. 이벤트도 선택된 프레임 시각까지로 한정한다. 프레임 이후에 발생한 처치나 오브젝트 획득을 먼저 포함하면 시점 제한이 무너지므로, 이벤트가 담긴 프레임과 개별 이벤트의 타임스탬프를 모두 확인한다.

입력 벡터는 파란 팀 합계에서 빨간 팀 합계를 뺀 골드, 경험치, CS, 챔피언 처치, 포탑 파괴, 드래곤 처치의 여섯 차이이다. CS 에는 라인 미니언과 정글 몬스터 수를 합산한다. 챔피언 처치는 처치자의 팀으로 집계하고, 포탑 파괴 이벤트의 teamId 는 파괴된 포탑의 소유 팀이므로 상대 팀에 실적을 부여한다. 드래곤은 killerTeamId 또는 처치자의 팀으로 귀속한다. 최종 KDA·최종 피해량·최종 골드·승패는 이 추출 함수의 입력 특성으로 읽지 않는다. 최종 승패는 학습 단계에서 정답을 만들 때만 사용한다.

학습 스크립트 train_model.py 는 동일한 extract_features 를 호출하여 학습과 서비스의 계산 기준을 맞춘다. 적격 경기 ID 의 중복을 제거하고 하나의 경기에서 파란 팀 관점의 표본 하나를 만든다. 같은 경기를 양 팀 관점으로 복제하여 서로 다른 분할에 넣지 않는다. 자료를 시작 시각 순으로 정렬한 뒤 대략 앞선 80%와 이후 20%로 나누며, 경계 시각이 같은 경기는 모두 평가 분할에 둔다. 최소 100 개 적격 고유 경기와 학습 60 개·평가 20 개 이상을 요구하고, 학습 자료에는 두 승패가 모두 있어야 한다. 실제 자료와 합성 자료를 하나의 학습·평가에 섞는 입력도 거부한다.

신경망 구조와 학습·저장

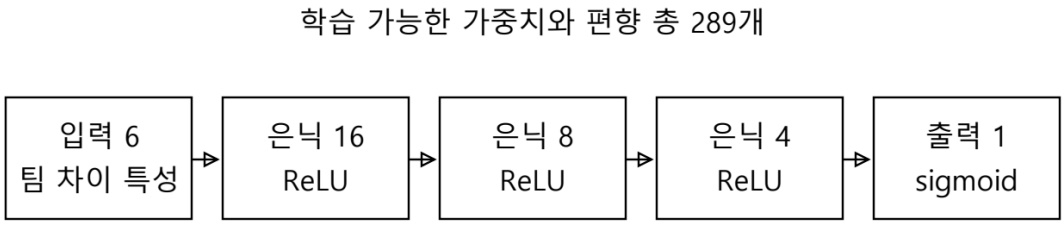


그림 2. 학습된 6-16-8-4-1 피드포워드 신경망 구조

제안 모형은 입력 6 개, 은닉층 16 개·8 개·4 개, 출력 1 개의 완전연결 다층 퍼셉트론이다. 학습 분할의 평균과 표준편차로 입력을 표준화하고 식 (1)~(3)에 따라 순전파한다. 여기서 $h_0=x$이고 $\mathrm{ReLU}(a)=\max(0,a)$이다. 세 은닉층은 ReLU, 마지막 층은 시그모이드다. 파란 팀의 추정치를 기준으로 빨간 팀에는 그 여사건 확률을 표시한다.

$$\tilde{x_j} = \frac{x_j - \mu_j}{s_j} \quad (1)$$

$$h_l = ReLU(W_l^{\mathrm{T}} h_{l-1} + b_l),\ l = 1,2,3 \quad (2)$$

$$p_B = \frac{1}{1+exp(-z)},\ z = W_4{}^{\mathrm{T}} h_3 + b_4 \quad (3)$$

학습 가능한 가중치와 편향은 첫 층 112 개, 둘째 층 136 개, 셋째 층 36 개, 출력층 5 개로 총 289 개이다. 세 비선형 은닉층을 실제로 학습하며, 시간별 모형도 같은 구조를 독립적으로 사용한다. 각 MLP 는 한 시점의 여섯 팀 차이만 받으므로 순환신경망의 시계열 기억을 구현한 것은 아니다. 과거 시점의 확률 변화 역시 실시간 코칭이나 사용자 실력 향상의 증거로 해석하지 않는다.

학습에는 scikit-learn 의 StandardScaler 와 MLPClassifier 를 사용한다. 표준화의 평균과 척도는 학습 자료에서만 적합하고 평가 자료에는 그대로 적용한다. MLP 의 활성화는 ReLU, 최적화는 L-BFGS, L2 정규화 계수 alpha 는 1.0 이다. random_state=42, max_iter=2000, max_fun=20000, tol=1e-6, early_stopping=False 로 고정하였다. 목적은 승패의 이진 log loss 와 정규화 항을 줄이는 것이다. dropout, 조기 종료 또는 평가 성능을 높이기 위한 구조·seed 탐색은 구현하지 않았다. 기존 실행 기록의 1,062 회는 최적화 반복 수이며 epoch 수로 부르지 않는다.

스크립트는 학습된 층별 가중치·편향, 특성 순서, 표준화 평균·척도, 모형 구조와 자료 출처를 일반 JSON 으로 내보낸다. 평가 보고서에는 입력 해시, 분할 경기 ID, 시각 범위, 패치·큐·승패 구성, 라이브러리 버전과 지표를 기록한다. 로지스틱 회귀와 학습 분할의 파란 팀 승리 비율을 쓰는 상수 기준선도 같은 평가 경기에서 비교한다. 이러한 정보는 결과의 계산 조건을 추적하기 위한 것이며, 이미 확인한 평가셋을 재사용한 이번 비교를 새로운 독립 검증으로 바꾸지는 않는다.

서비스는 _read_model 과 공통 검증 함수에서 2MB 크기 상한, 스키마, 특성 순서와 출처를 검사한다. 은닉층과 활성화, 289 개 매개변수, 인접 층의 차원과 유한 수치, 양수

표준화 척도를 확인한다. 공통 순전파는 표준화와 층별 중간 결과에도 유한성 검사를 적용하고 마지막 시그모이드 입력을 −60~60 으로 제한한다. 주 추정, 입력 대체 민감도와 시점별 추론이 검증된 계산을 재사용한다. 손상·누락·출처 불일치는 임의 확률 대신 이용 불가로 처리하며, 실행 가능한 pickle 이나 서비스 중 재학습을 사용하지 않는다.

분석 응답과 사용자 화면의 연결

현재 분석 응답은 기존 통계와 모형·프로필에 native, loadout, tips, pagination, ingestion 및 dataset 정보를 더한다. 구매·스킬 기록의 누락과 모호한 되돌리기를 명시하며, 수집·집계 버전이 일치하는지 확인할 수 있다. Riot 목록 시작 위치는 0~10000, 한 요청의 경기 수는 1~20 으로 제한한다. 중단된 페이지는 첫 미처리 ID 부터 다시 조회한다. 화면의 이전·다음 버튼은 이미 표시한 플레이어·지역·모드를 사용하므로 입력란을 수정해도 다른 플레이어의 전적과 섞이지 않는다.

대시보드는 최근 표본의 승률, 전체 KDA, 평균 CS/min 과 킬 관여율을 보여 주며 경기 선택과 승·패 필터를 제공한다. KDA 는 전체 처치·도움 합계를 전체 사망 수로 나누고 분모를 최소 1 로 두며, 경기별 CS/min 은 미니언·몬스터 수를 경기 분으로 나눈다. 킬 관여율은 처치·도움을 팀 처치 수로 나누고 0~1 범위에서 계산한다. 선택 경기에는 시야 점수, 챔피언 피해량, 사용자 팀에서 상대 팀을 뺀 골드 차이 그래프, 초기 확률 추정과 실제 결과를 함께 배치한다. 전체 경기 그래프는 초기 시점 이후 자료를 포함하므로 신경망 입력과 구분하여 읽어야 한다.

경기 후 검토는 관측 통계에 적용한 규칙과 자기 기록 비교를 사용한다. 사망 직전의 위치나 판단 의도는 집계 통계만으로 확인할 수 없으므로 인과 설명을 하지 않는다. 모형

화면은 주 신경망의 구조·학습 진단·평가 지표와 추가 Random Forest 비교를 보여 준다. 시간별 추정에도 각 시점의 평가 수와 기준선을 표시하고, 입력 대체 결과를 인과 기여도로 부르지 않는다. 출처 배너·경고·예측 불가 상태를 유지하며 분석 JSON 을 내보낼 수 있다.

표 1. 변경하지 않은 실제 경기 표본의 구성

분할	경기 수	파란 팀 승리	빨간 팀 승리
학습	83	48	35
평가	21	10	11
전체	104	58	46

■ 구현 및 결과분석

1. 구현 범위와 분석 흐름

LolAi.gg 는 League of Legends 의 완료된 경기 기록을 수집하여 최근 경기의 통계와 복기 자료를 제공하고, 경기 초반의 팀 간 차이로 최종 승리 확률을 추정하는 연구용 웹 애플리케이션이다. 구현 결과는 화면 기능의 동작, 학습된 신경망의 추론, 예측 성능을 구분하여 판단하였다. 화면에 확률이 표시된다는 사실만으로 높은 정확도나 이용자의 실력 향상이 입증되는 것은 아니므로, 결과에는 각 검증이 확인한 범위를 함께 제시하였다.

분석은 입력 검증, 경기·타임라인 조회, 적격 경기 선별, 통계와 개인 집계, 관측별 특성 추출, 저장 모형 추론과 화면 표시로 이어진다. 최종 KDA·분당 CS·킬 관여율과 자기 기록 비교는 사후 복기용이다. 주 추정과 시간별 곡선은 각각의 관측 프레임까지만 사용한다. 전체 경기 골드 흐름, 최종 기록 또는 현재 계정의 랭크·숙련도를 과거 예측 입력에 혼용하지 않는다.

2. 실제 데이터와 전처리 구현

학습과 평가에는 인증된 Riot 응답으로 확보된 고유 경기·타임라인 쌍 104 개를 사용하였다. 이 자료는 한국 서버의 소환사의 협곡 랭크 개인/2 인 경기이며, 수집 범위는 2026 년 10 월 20 일부터 28 일까지, 패치는 16.20 과 16.21 이다. 한 경기를 블루팀 기준 한 행으로 구성하였고 동일 경기의 양 팀을 별도 표본으로 중복 계산하지 않았다. 다만 경기 고유성이 참가자 독립성을 뜻하지는 않는다. 수집은 하나의 시작 계정과 관측된 소수 참가자에서 확장되었으므로 같은 이용자가 여러 경기에 포함될 수 있다.

특징은 블루팀에서 레드팀을 뺀 골드, 경험치, CS, 킬, 타워, 드래곤의 여섯 가지 차이이다. 팀 식별은 참가자 번호를 임의로 나누는 방식 대신 경기의 팀 메타데이터를 사용한다. CS 는 미니언과 정글 몬스터 처치를 합산하며, 타워 파괴 이벤트의 팀 정보는 파괴한 팀이 아닌 파괴된 타워의 소유 팀이라는 점을 반영한다. 이러한 처리는 단순한 합계 계산에서도 방향이 뒤집히거나 일부 이벤트가 잘못 집계되는 오류를 막기 위한 것이다.

관측 시점은 9 분 이상 10 분 이하 구간의 가장 늦은 완전한 프레임으로 제한하였다. 선택한 프레임에 참가자 열 명의 정보가 모두 있어야 하며, 이벤트도 해당 프레임의 시각까지만 반영한다. 현재 자료의 실제 프레임은 약 540.09 초로, 다음 프레임이 600 초를 조금 넘기 때문에 제외되었다. 따라서 본 실험을 정확히 600 초 시점에서 측정한 결과로 표현하지 않는다. 최종 골드·최종 킬 등 종료 통계와 이후 이벤트는 입력에 포함하지 않고, 최종 승패는 지도학습의 정답으로만 사용한다.

104 경기는 시간순으로 앞선 83 경기를 학습용, 뒤의 21 경기를 평가용으로 나누었다. 학습 자료는 블루 승리 48 경기와 레드 승리 35 경기이고, 평가 자료는 각각 10 경기와 11 경기이다. 두 구간의 경기 ID 는 겹치지 않으며 학습 경기 전부가 종료된 뒤에 첫 평가 경기가 시작되는 경계도 확인하였다. 이는 단순히 시작 시각만 정렬했을 때 마지막 학습 경기와 첫 평가 경기가 진행 시간상 겹칠 수 있는 문제를 점검한 것이다.

표준화는 $z_j=(x_j-\mu_j)/s_j$ 로 계산하며, 평균 μ_j 와 척도 s_j 는 학습 구간에서만 구하였다. 평가 데이터까지 이용해 평균이나 분산을 맞추면 미래 구간의 분포가 전처리에 반영되므로 이를 허용하지 않았다. MLP 와 로지스틱 회귀는 같은 특징, 같은 경기 분할, 같은 정답과 표준화 값을 사용하였다. 이 조건은 모델 간 비교에서 전처리와 표본 차이가 성능 차이의 원인이 되는 일을 줄이지만, 표본의 대표성까지 보장하지는 않는다.

3. 다층 신경망과 추론 구현

딥러닝 모델은 입력층 6 개, 은닉층 16·8·4 개, 출력층 1 개의 다층 퍼셉트론으로 구현하였다. 구조는 6→16→8→4→1 이며 은닉층은 ReLU, 출력층은 시그모이드 함수를 사용한다. 각 은닉층은 $h=\mathrm{ReLU}(Wx+b)$, 출력은 $p=1/(1+\exp(-z))$로 계산한다. 세 은닉층의 비선형 변환을 실제 데이터로 학습하므로, 단일 선형식에 시그모이드를 적용하는 로지스틱 회귀와 구별되는 신경망 구현이다.

학습 가능한 가중치와 편향은 $(6\times16+16)+(16\times8+8)+(8\times4+4)+(4\times1+1)=289$ 개이다. 학습에는 scikit-learn 의 MLPClassifier 와 L-BFGS 최적화를 사용하고, 정규화 계수 alpha=1, 난수 시드 42, 최대 반복 2,000 회, 최대 함수 호출 20,000 회, 허용오차 10^{-6}으로 고정하였다. 조기 종료는 사용하지 않았다. 구조와 설정은 이번 비교 실행 전에 정했으며, 평가 점수를 높이기 위한 구조·시드·정규화 계수 탐색은 수행하지 않았다. 저장 보고서에는 1,062 회 반복과 수렴 상태가 기록되어 있고 수렴 경고는 없었다.

학습 결과는 표준화 값, 층별 가중치·편향, 모델 종류와 평가 정보를 포함한 JSON 으로 저장한다. 서비스의 추론은 이 파일을 읽어 Python 표준 라이브러리로 순전파를 계산하므로 실행 가능한 직렬화 객체를 불러올 필요가 없다. 출력은 블루 승리 확률이며 레드팀을 조회하면 $1-p$ 로 변환한다. 파일 형식이나 차원이 잘못되거나 비유한 수치·중간 연산 오버플로가 발생하면 확률 제공을 중단한다. 실제 경기와 합성 자료의 출처가 다를 때도 추정을 제공하지 않아 시연용 모델이 실제 성능처럼 표시되는 것을 막는다.

모형 교체 후 과거 확률이나 추가 비교 결과가 남는 문제를 줄이기 위해 주 모형, 시간별 모형 묶음과 비교 보고서의 SHA-256 을 분석 캐시에 반영한다. 가중치뿐 아니라 평가

정보가 바뀌어도 별도 버전으로 취급하며 저장 직전에도 확인한다. 기존 Riot 원응답은 재사용하되 구버전 분석을 새 모형의 결과로 반환하지 않는다. 이는 예측 품질과 별도로 화면과 현재 모형의 대응을 유지하는 구현이다.

4. 평가 지표와 비교 조건

정확도는 전체 n 경기 중 예측 승패가 실제 승패와 일치한 비율로, 식 (4)로 계산한다. 본 비교에서는 블루 승리 확률을 0.5 기준으로 이진화한다. 정확도는 맞힌 경기 수를 쉽게 보여주지만 0.51 과 0.99 를 같은 승리 예측으로 처리하므로 확률의 품질을 충분히 설명하지 못한다. 평가 표본이 21 개이므로 한 경기의 차이만으로도 정확도가 약 4.76%포인트 변한다는 점도 함께 고려해야 한다.

$$Accuracy = \frac{1}{n}\sum_{i=1}^{n} I(\hat{y}_i = y_i) \quad (4)$$

ROC-AUC 는 임계값 하나에서의 정답률 대신 승리 경기의 예측 점수가 패배 경기보다 높게 배치되는 정도를 측정한다. 승리 표본 수를 n_+, 패배 표본 수를 n_-라고 하면 $AUC=[\Sigma I(p_+>p_-)+0.5\Sigma I(p_+=p_-)]/(n_+n_-)$로 표현할 수 있다. 이 값은 순위 구별 능력의 지표이며, 높은 값이 곧 확률 보정을 의미하지는 않는다. 평가 자료에는 두 클래스가 모두 있으므로 산출할 수 있었고, 단일 클래스일 때는 계산 불가능한 값을 임의로 만들어 내지 않도록 구현하였다.

Brier 점수는 식 (5)로 계산하는 확률의 평균 제곱오차이며 작을수록 좋다. 실제 승리 정답을 1, 패배를 0 으로 두므로 자신 있게 틀린 예측에 큰 오차가 부여된다. 따라서 정확도가 같아도 Brier 점수는 다를 수 있다. 이 점수는 확률 예측 전체의 오차를 평가하지만, 하나의 Brier 값만으로 모든 확률 구간의 보정이 검증되었다고 결론내리지는 않았다.

$$Brier = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2 \quad (5)$$

초기 MLP 실험의 비교 대상은 로지스틱 회귀와 학습 파란 팀 승리 비율을 항상 출력하는 상수 기준선이다. 상수 확률은 48/83, 약 0.578313 이며 평가 구간의 비율을 사용하지 않았다. 모든 경기의 점수가 같아 AUC 는 0.5 이다. 표 2 와 그림 3·4 는 이 초기 비교를 보존한다. 같은 분할의 추가 Random Forest 결과와 시간별 모형의 별도 평가 조건은 후술한다. 어느 모형도 정확도 하나만으로 우수성을 판정하지 않는다.

5. 측정 결과와 해석

표 2. 보존한 초기 MLP 실험의 동일 21 경기 비교

모형	정확도	ROC-AUC	Brier	정답 수
딥 MLP	47.62%	0.49091	0.372683	10/21
로지스틱 회귀	57.14%	0.60909	0.263561	12/21
상수 기준선	47.62%	0.50000	0.259862	10/21

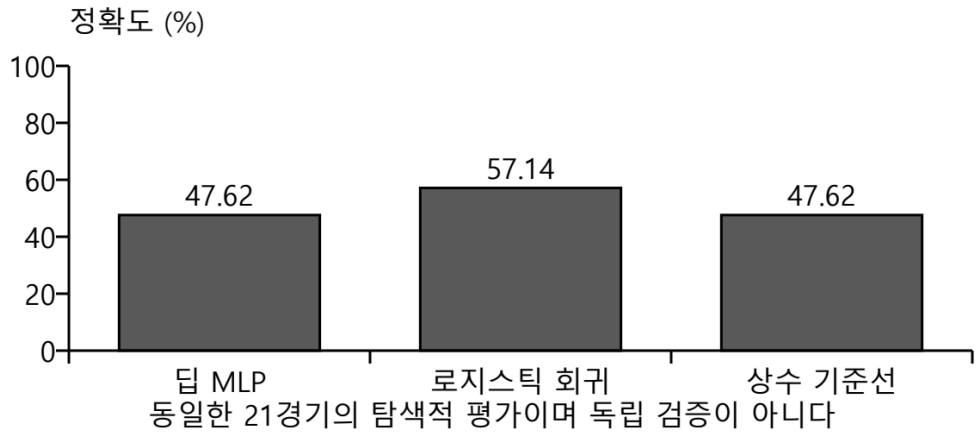


그림 3. 초기 MLP 실험과 두 기준선의 정확도

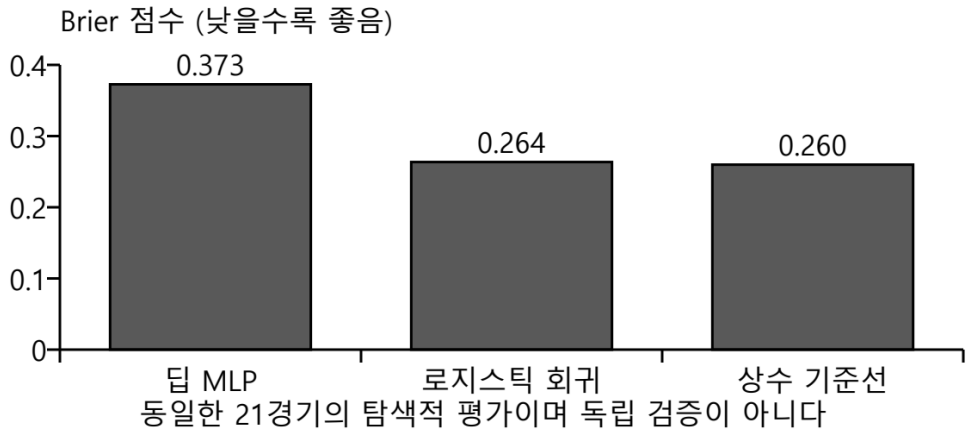


그림 4. 초기 MLP 실험과 두 기준선의 확률 오차

학습 구간에서 MLP 는 83 경기 중 79 경기를 맞혀 정확도 95.18%, ROC-AUC 0.995833, Brier 0.034691 을 기록하였다. 이 수치는 이미 학습에 사용한 자료에 대한 적합도이며 검증 성능이 아니다. 같은 모델을 이후 21 경기에 적용한 결과는 10 경기 정답, 정확도 47.62%, ROC-AUC 0.490909, Brier 0.372683 이었다. 학습 점수와 평가 점수의 큰 격차는 현재 작은 표본에서 과적합이 발생했을 가능성과 일치한다.

동일한 평가 구간에서 로지스틱 회귀는 12 경기를 맞혀 정확도 57.14%, ROC-AUC 0.609091, Brier 0.263561 을 기록하였다. 상수 기준선은 10 경기 정답, 정확도 47.62%, AUC 0.500000, Brier 0.259862 였다. MLP 는 정확도에서 상수 기준선과 같고, AUC 와 Brier 에서는 더 나빴다. 로지스틱 회귀도 정확도와 AUC 는 높았지만 Brier 는 상수 기준선보다 컸으므로, 이 표본에서 확률 오차가 개선되었다고 말할 수 없다.

로지스틱 회귀와 MLP 의 정답 수 차이는 두 경기, 정확도 차이는 약 9.52%포인트이다. 작은 평가 표본의 이 차이만으로 통계적으로 유의한 우열이나 일반적인 모델 순위를 주장하지 않았다. 또한 AUC 가 0.5 부근이라는 이유로 모든 조건에서 무작위 예측과 동일하다고 단정할 수 없다. 확인한 사실은 이번에 고정한 자료와 분할에서 얻은

점수이며, 새로운 참가자나 다른 패치에서도 같은 관계가 유지되는지는 별도 자료로 검증해야 한다.

신경망의 부정적 결과는 구현 실패와 구분된다. 학습된 가중치가 존재하고 저장 후 순전파가 재현되더라도 일반화 성능은 낮을 수 있다. 83 개 학습 표본에 비해 289 개 매개변수를 가진 모델은 자료의 작은 패턴에 민감할 수 있고, 정규화를 적용했다는 사실만으로 이를 해결했다고 볼 수 없다. 현재 결과에서 중요한 성과는 신경망을 만들었다는 사실과 그 한계를 측정 가능한 수치로 공개했다는 점이다. 딥러닝이라는 모델 종류를 근거로 성능 향상을 전제하지 않았다.

평가 구간은 이전 로지스틱 회귀 결과에서 이미 확인한 21 경기를 다시 사용하였다. 이번 신경망 설정을 점수에 맞춰 반복 탐색하지 않았더라도, 이미 본 평가 자료를 새로운 독립 시험 자료라고 부를 수는 없다. 따라서 이번 비교는 탐색적 실험으로 한정한다. 수집 경로의 연결된 참가자 집단, 좁은 수집 기간, 두 패치의 차이도 결과에 영향을 줄 수 있으나 각 요인의 기여를 분리하여 측정하지 않았다. 과적합과 분포 변화 중 어느 하나를 단독 원인으로 확정하지 않는다.

6. 재현성·검증 결과와 남은 범위

저장된 JSON 매개변수를 이용한 독립적인 표준 라이브러리 순전파 계산은 재학습 없이 평가 지표를 10^{-12} 이내 오차로 재현하였다. 이 과정에서는 입력 자료 해시, 학습·평가 분할, 표준화 값, 매개변수 289 개, 팀 방향과 시간 경계도 확인하였다. 이는 저장 파일과 보고된 수치의 일관성을 뒷받침한다. 다만 같은 자료를 다시 계산한 재현 확인이므로 외부 표본에서의 성능 검증이나 신뢰구간을 대신하지는 않는다.

현재 모델 테스트는 미래·종료 통계를 바꾸어도 초반 특징이 변하지 않는지, 불완전한 프레임과 잘못된 경기 대응을 거부하는지, 팀 방향과 타워 소유자를 올바르게 처리하는지 점검한다. 중복 제거와 시간순 분할, 학습 자료만 사용하는 표준화, 실제·합성 출처 불일치, 손상된 모델, 비유한 값과 큰 수치, 중간 연산 오버플로도 다룬다. 신경망 부분은 가상의 120 개 검증용 레코드에서 scikit-learn 추론과 JSON 순전파의 일치, 레드 확률의 보수 관계, 두 모델의 동일 분할·전처리를 확인한다. 이 가상 자료의 테스트를 실제 경기 예측 성능으로 계산하지 않았다.

이전 0.2.0 전체 검사는 135 개와 하위 검사 6 개가 12.78 초에 통과한 역사적 기록이다. 최종 0.6.0 에서는 정상 Cursor 터미널에서 276 개와 하위 검사 6 개가 32.90 초에 통과하였다. 기존 Starlette 의존성 경고 1 개는 비차단 안내였고 의존성을 임의 교체하지 않았다. 프런트엔드 41 개 검사, TypeScript 와 Vite 빌드도 통과하였다. 기능 검사와 실제 API 확인은 예측 품질이나 운영 환경 전체를 보증하지 않는다.

이전 0.2.0 의 실제 10 경기 JSON 에서는 주 확률 10 개와 시간별 확률 110 개를 설치된 모형으로 재계산하였다. 이후 0.4.1~0.6.0 의 브라우저 검사에서는 실제 랭크 기록과 OP.GG 아카이브, 수집 중복 방지·페이지 이동·구매 및 스킬 이력과 저장된 추가 평가를 확인하였다. 각 단계의 로그와 평가 표본은 별도로 유지한다. 현재 계정 상태나 영상 응답을 과거 예측 입력으로 사용하지 않는다.

자료 허브는 선택 경기의 챔피언·역할과 경기 패치, 별도로 조회한 Data Dragon 버전을 보여 준다. OP.GG·FOW·GGQ·DEEPLOL·YOUR.GG 의 공식 사이트 및 YouTube 검색 링크에는 플레이어 식별자를 넣지 않는다. 선택적 YouTube Data API 경로는 백엔드 키로 실제 영상의 채널·게시일·조회수를 가져와 표시하고, 패치 문자열의 텍스트 일치

여부를 구분한다. 잘못된 패치 입력에는 조회 링크나 API 요청을 만들지 않으며, 영상 요청도 사용자가 버튼을 누를 때만 수행한다.

영상의 조회수·게시일·패치 문자열은 교육 품질의 보증이 아니다. 자체 전적·챔피언·빌드 집계, 복기 공식과 개인 목표 추적은 구현했으나 조언의 인과적 효과는 확인하지 못했다. OP.GG 공식 MCP 는 후속 단계에서 실제 스키마·빌드 응답과 과거 시즌 랭크 아카이브를 확인하였다. FOW·GGQ·DEEPLOL·YOUR.GG 의 전체 데이터나 내부 알고리즘을 복제·통합한 서비스로 주장하지 않는다.

작성 근거: docs/DEEP_LEARNING_RESULTS.md, docs/TEMPORAL_MODELS.md, docs/QA_FEATURE_EXTENSION_2026-10-30.md, docs/TEST_RESULTS.txt, 관련 모형·프로필·연동 소스 및 저장된 집계 보고서. 본 절에는 원시 참가자 기록이나 인증 정보를 전재하지 않았다.

7. 개인별 복기와 시간별 모형 확장

profiles.py 는 불러온 적격 경기를 중복 없이 집계하여 다섯 역할과 챔피언별 경기 수·승률·KDA·분당 CS·킬 관여율·시야를 제공한다. 관측이 없는 역할의 승률은 0%로 만들지 않고 비어 있는 값으로 표시한다. 표본에서 가장 많이 한 역할과 다른 역할을 비교하되 동률과 역할 미상은 구분한다. 이는 최근 표본의 기술통계이며 주력 역할 선호, 자동 배정 여부나 장기 실력을 알아낸 결과가 아니다.

선택 경기의 개인 비교는 그 경기를 제외한 같은 역할의 다른 경기 세 개 이상이 있을 때 제공한다. 같은 챔피언·역할의 표본이 충분하면 그 집단을 우선한다. 다른 경기에는 선택 경기보다 이후에 끝난 기록도 있을 수 있으므로 사후 복기에만 사용하고 예측 입력으로 넣지 않는다. 세 경기는 표시를 위한 최소 조건이며 통계적 검증의 기준이

아니다. 제어 와드 수와 항복 표시는 원응답에 있을 때만 제공하고 AFK 나 고의성으로 해석하지 않는다.

시간별 모형은 기존 104 경기의 전역 학습 83 개·평가 21 개 배정을 고정한 뒤, 목표 5~20 분마다 유효한 관측을 따로 추출하였다. 각 시점의 학습 자료로만 표준화하고 동일한 6-16-8-4-1 MLP 를 고정 설정으로 학습하였다. 최소 학습 60 경기·평가 20 경기를 요구한 결과 5~15 분 목표의 11 개 모형을 얻었다. 16~20 분 목표는 평가 표본 부족으로 제외하였다. 실제 프레임은 약 4~14 분이므로 그래프의 가로축에는 목표 명칭 대신 관측 시각을 반영한다.

5~10 분 목표는 학습 83·평가 21 개, 11 분은 82·21 개, 12~15 분은 82·20 개를 사용하였다. 늦은 시점에는 아직 종료되지 않은 경기만 남으므로 시점 간 평가 모집단도 같다 할 수 없다. 모든 시간별 지표는 이미 확인한 평가셋의 탐색적 재사용 결과이며, 가장 좋은 시점을 골라 전체 정확도로 제시하지 않았다. 11 개 중 10 개 시점에서 MLP 의 Brier 가 로지스틱 회귀보다 컸고, 10 개 시점에서 상수 기준선보다 컸다. 독립 NumPy 순전파가 227 개 평가 시점의 확률과 지표를 재현했지만 이들은 같은 20~21 경기의 반복 관측이다.

입력 민감도는 한 번에 여섯 특성 중 하나를 학습 평균으로 바꾸고 나머지를 고정한 채 같은 모형을 다시 실행하여 계산한다. 현재 사용자 팀 확률에서 대체 입력의 확률을 뺀 값을 표시하며, 입력값 자체는 파란 팀에서 빨간 팀을 뺀 방향을 유지한다. 이 차이는 SHAP 이나 합이 맞는 기여도, 행동의 인과효과가 아니다. 서로 연관된 특성을 하나씩 바꾸면 실제 경기에서 드문 조합이 될 수도 있다. 관측된 처치·포탑·억제기·주요 몬스터 이벤트는 복기 시각을 제공할 뿐 결정적 패배 원인이나 한타를 추정하지 않는다.

Random Forest 는 기존 주 모형을 교체하지 않고 같은 입력·학습 분할·평가 분할에서 별도로 비교하였다. 트리 200 개, 최대 깊이 5, 잎의 최소 표본 3, 난수 시드 42 로 고정했으며 평가 점수에 맞춘 탐색이나 최종 모형 선택을 하지 않았다. 다만 이 설정은 평가셋을 이미 본 뒤 정했으므로 사후 탐색적 비교라는 한계가 있다. 정확도 66.67%와 ROC-AUC 0.59091, Brier 0.260681 을 기록하였다. 정확도가 높아도 AUC 는 로지스틱 회귀보다 낮고 Brier 는 상수 기준선보다 커서 우수성을 확정할 수 없다. XGBoost 는 현재 환경에 설치되지 않아 평가하지 않았으며 과거 79% 기록의 재현으로 취급하지 않는다.

표 3. 같은 분할에서 수행한 추가 Random Forest 비교

모형	정확도	ROC-AUC	Brier	정답 수
Random Forest	66.67%	0.59091	0.260681	14/21
로지스틱 회귀	57.14%	0.60909	0.263561	12/21
상수 기준선	47.62%	0.50000	0.259862	10/21

요청 시의 현재 랭크·챔피언 숙련도는 Riot 의 플랫폼 API 에서 조회하고 최대 5 분 캐시한다. 실제 계정에서 랭크와 선택 챔피언 숙련도 응답을 확인했으나, 이를 과거 학습 경기 당시의 값이나 검증된 전문성으로 소급하지 않는다. YouTube 는 최근 14·30·90 일 중 기간과 조회수·최신순을 선택하여 검색하고 별도 상세 요청으로 채널·게시일·조회수를 받는다. Ahri·Mid·패치 16.21 의 최근 30 일 조회에서 실제 영상 한 개를 표시했지만 텍스트에 해당 패치가 없다는 사실도 화면에 표시하였다. 영상의 실제 플레이 버전과 조언 품질은 검증하지 않았다.

OP.GG 의 공식 인터페이스는 선택 보조 자료로 사용하며 출처와 수집 시각을 보존한다. Riot API 와 LolAi 자체 계산이 주 분석 경로이다. 과거 시즌 자료는 제공사가 보고한

season ID 와 랭크이며 달력 연도나 실제 경기 당시 티어를 임의로 추정하지 않는다. 비공개 의견, 전향적 저티어 평가 코드와 모형 이후 추가 표본 점검은 구현 범위와 실제 증거를 각각 구분한다.

8. 자체 분석과 검증 확장

최종 0.6.0 은 외부 사이트의 화면이나 내부 코드를 복제하지 않고 Riot 원응답을 자체 계산한다. 10 명 전적표, 팀 피해·골드 비중, 같은 역할 상대와 9~10 분 및 14~15 분 차이, 실제 오브젝트 처치·도움 기록과 사망 시각을 제공한다. 기록된 관여와 시간 간격은 책임·회피 가능성 또는 승패 원인을 입증하지 않는다.

새 실제 조회는 경기 ID 를 기본키로 별도 SQLite 에 저장한다. 더 완전한 타임라인은 기존 기록을 보완하고 중복 경기는 수를 늘리지 않는다. 챔피언 집계와 조언 참고 표본은 한 트랜잭션으로 함께 게시한다. 원래 104 경기 JSONL 과 학습 모형은 보존하며 합성 데모는 저장 집단에서 제외한다. 실패한 입력과 갱신은 기존 자료를 훼손하지 않도록 처리한다.

추가 시기 점검을 위해 MLP 와 로지스틱 모형의 원본 해시·가중치 사본, 이미 본 108 경기 제외 목록을 고정하였다. 한 실제 경기의 다른 참가자들을 정해진 순서로 선택하여 각 최근 10 경기를 수집하였다. 최초 세 명에서 적격 표본 8 개만 얻어, 예측 지표를 계산하기 전에 나머지 여섯 명까지 한 번 확장하였다. 전적 수집 결과 180 개 고유 경기를 확보했고, 108 개 기존 경기와 모형 생성 전 37 개를 제외한 35 개를 점검했다. 선정 규칙은 성능이 좋은 경기만 고른 절차가 아니지만 회고적 편의표본이라는 한계가 있다.

추가 35 경기는 파란 팀 승리 15·빨간 팀 승리 20 경기였다. 모형 생성 시각 이후 경기만 포함하며, 원래 학습·평가 경기와 이미 서비스에서 확인한 네 경기는 제외했다. 한 경기당

파란 팀 한 행으로 계산했고 재학습·튜닝·활성 모형 교체는 하지 않았다. 316 명의 참가자 중 원래 스냅샷에도 있던 참가자는 3 명이지만, 모집 방식과 같은 경기 관계가 독립 외부 검증을 보장하지는 않는다.

MLP 는 20/35, 로지스틱 회귀는 28/35 를 맞혔다. 상수 기준선은 원래 학습 표본의 파란 팀 승리 비율 48/83 을 유지하여 15/35 를 맞혔다. 표 4 는 같은 추가 표본의 결과이며 앞선 표 2·3 의 21 경기 결과와 분리한다. 신경망의 Brier 는 상수보다 높아 확률 품질의 개선을 주장하지 않는다.

표 4. 변경하지 않은 모형의 추가 35 경기 점검

모형	정확도	ROC-AUC	Brier	정답 수
MLP	57.14%	0.670000	0.319481	20/35
로지스틱	80.00%	0.873333	0.168850	28/35
상수	42.86%	0.500000	0.267321	15/35

loadout_timeline.py 는 ITEM_PURCHASED, ITEM_SOLD, ITEM_DESTROYED, ITEM_UNDO 를 시각순으로 보존한다. 알려진 직전 구매·판매와 일치한 되돌리기만 취소 처리한다. 원래 구매 이력에는 이후 판매·조합·소모된 아이템도 남으며 최종 인벤토리와 구분한다. 항목을 특정할 수 없는 되돌리기나 잘못된 이벤트가 있으면 구매 집계에 넣지 않는다. 일반 SKILL_LEVEL_UP 의 Q/W/E/R 선택은 따로 정리하고 진화 이벤트와 누락을 임의 보충하지 않는다.

자체 챔피언 집계는 챔피언·역할·패치·대전 유형별로 최종 인벤토리와 관측된 첫 여섯 구매·스킬 선택을 구분한다. 180 경기 초기 집계에서 구매 순서 사용 가능 플레이어-경기

행은 1749 개, 스킬 행은 1800 개였다. 사용 가능 행 수는 고유 경기 수가 아니다. 순서 빈도의 분모도 해당 기록을 사용할 수 있는 행이며 최적 빌드나 효능을 뜻하지 않는다.

tips_engine.py 의 우선순위는 gap × equal weight × consistency 이다. gap 은 성능 백분위가 목표 50 백분위에 못 미친 비율이고 consistency 는 최근 경기일수록 0.85 의 거듭제곱으로 더 높은 비중을 주는 하위 비율이다. 유효 지표에 동일 가중치를 나누므로 학습된 승리 영향력으로 부르지 않는다. 최대 세 가지 복기 우선순위와 상대 강점을 실제 수치로 제시한다.

참고 집단은 같은 역할·패치·대전 유형으로 제한하고 본인과 현재 불러온 경기를 제외한다. 각 지표는 별도 30 경기와 서로 다른 플레이어 10 명 이상, 최근 경기 3 개 이상을 요구한다. 모두 같은 챔피언이고 별도 50 경기 이상이면 챔피언별 참고를 사용한다. 표본 수와 지표 관측 비율을 함께 기록하며, 작은 반복 표본을 통계적 확실성으로 표현하지 않는다. 검증된 경기 전 티어가 없으면 티어별 참고는 활성화하지 않는다.

개인 연습 추적은 CS/min·시야/min·킬 관여율·사망 중 사용자가 선택한 목표를 저장한다. 서버는 같은 역할·패치·대전 유형의 이전 완료 경기 3~10 개로 기준 평균과 ID 목록을 고정한다. 저장 시각보다 이후에 시작한 첫 3 개 경기만 추적하고 목표 달성과 평균 차이를 계산한다. 페이지를 다시 열어도 로컬 SQLite 에 남지만 변화가 조언의 인과적 효과를 입증하지는 않는다.

현재 Riot 랭크는 조회 날짜별로 저장하고 캐시 응답을 새 관측으로 중복 기록하지 않는다. 선택적 OP.GG 아카이브 API 는 실제로 제공된 과거 시즌 항목을 날짜·출처와 함께 저장한다. 예를 들어 제공사가 보고한 season 31 의 Silver III 38LP 기록을

확인했지만 시즌 ID 를 임의 연도로 바꾸지 않는다. 현재 랭크나 외부 아카이브를 예전 경기의 확정 티어로 사용하지 않는다.

check.cmd 의 정상 터미널 결과는 276 passed, 1 warning, 6 subtests passed in 32.90s 였다. 프런트엔드 검사 41 개와 TypeScript·Vite 빌드도 통과하였다. 재시작한 0.6.0 의 집계·이력·평가 API 와 페이지 이동을 별도로 확인한다. 단위·통합·실제 경로 검사를 기능 완성의 근거로 기록하되 공개 운영의 부하·보안 검증이나 예측 우수성으로 확대하지 않는다.

백업 도구는 원시 JSONL, 저장 모형과 연구 보고, 수집·랭크·비공개 노트 SQLite 를 별도 폴더에 복사하고 SHA-256 및 SQLite 무결성을 확인한다. 실행 중 데이터베이스는 SQLite backup 기능을 사용한다. API 키와 응답 캐시는 제외하며, 이 자료는 비공개로 보관해야 한다. 같은 컴퓨터의 백업만으로 분실에 대비할 수 없으므로 외부 저장소 복사는 별도 확인이 필요하다.

저티어 검증 스크립트는 동의한 플레이어의 현재 랭크를 등록한 뒤 이후 경기만 평가하도록 구현하였다. 현재 적격 신규 저티어 코호트는 없으며 성능 숫자를 만들지 않는다. 동료 의견은 비공개 미인증 기록이다. 동료 본인 인증·효과 검증과 실시간·음성 코칭, 통계 기반 추정 실력은 이번 연구용 MVP 의 완성 범위로 주장하지 않는다.

■ 결론 및 소감

결론

LolAi.gg 는 완료 경기 수집부터 자체 전적표·상대 비교·챔피언과 구매 이력 분석·설명 가능한 복기·개인 목표 추적·저장 신경망 추론까지 연결하였다. 276 개 백엔드 검사와 실제 Riot·OP.GG·YouTube 경로 확인은 기능 구현의 근거이다. 원래 104 경기 실험을 보존하면서 별도 수집·추가 평가를 수행하여 데이터 확장과 모형 결과의 출처를 구분하였다.

원래 21 경기와 추가 35 경기 모두에서 신경망의 예측 우수성은 입증되지 않았다. 추가 표본의 MLP Brier 0.319481 은 상수 기준선 0.267321 보다 높았고 로지스틱 회귀 0.168850 보다도 컸다. 표본을 늘려 다른 결과가 나왔다는 사실을 같은 모형의 개선으로 부르지 않는다. 본 작품의 성과는 분석 시스템 구현과 재현 가능한 비교, 그리고 작은 신경망의 과적합·확률 오차와 자료 한계를 확인한 데 있다.

추가 표본은 편의표본이며 모집·평가 규칙도 회고적으로 정하였다. 모델 가중치와 제외 목록을 추가 수집 전에 고정했지만 독립 외부 모집단 검증이나 무작위 코칭 실험은 아니다. 낮은 티어의 전향적 표본과 동료 본인 인증, 조언의 효과는 별도 연구가 필요하다. 기존 파일의 백업과 현재 검토본은 보존했으며, 학교 제출과 지도교수의 확인은 별도 절차이다.

개인 소감 및 기여 확인

이전에 로컬에서 구현했던 프로그램과 자료를 잃은 뒤 현재 환경에서 재구성하였다. 기억에 남은 과거 결과와 이번에 파일·로그로 확인할 수 있는 결과를 분리하는 일이

중요했다. AI 코딩 도구의 도움을 받아 기능과 검사를 확장했고, 정상 터미널에서 전체 검사를 실행해 동작을 확인하였다. 예측 숫자 하나를 보여 주는 것보다 자료의 출처, 조언이 나온 근거, 실제로 확인하지 못한 한계를 설명해야 한다는 점을 경험하였다.

■ 참고문헌

[1] 김철기, 이수원. (2020). 양방향 순환신경망 임베딩을 이용한 리그오브레전드 승패 예측. 정보처리학회논문지.소프트웨어및데이터공학, 9(2), 61-68. DOI (https://doi.org/10.3745/KTSDE.2020.9.2.61).

[2] 김용우, 김영민. (2021). 기계학습을 활용한 게임승패 예측 및 변수중요도 산출을 통한 전략방향 도출. 한국게임학회 논문지, 21(4), 3-12. DOI (https://doi.org/10.7583/JKGS.2021.21.4.3).

[3] 김예지, 민정혜. (2023). AOS 게임 유저 데이터를 이용한 승패 예측. 한국컴퓨터정보학회논문지, 28(12), 23-32. DOI (https://doi.org/10.9708/jksci.2023.28.12.023).

[4] Y. LeCun, Y. Bengio, G. Hinton. (2015). Deep learning. Nature, 521, 436-444. DOI (https://doi.org/10.1038/nature14539).

[5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research, 15(56), 1929-1958. Publisher (https://www.jmlr.org/papers/v15/srivastava14a.html).

[6] A. L. C. Silva, G. L. Pappa, L. Chaimowicz. (2018). Continuous Outcome Prediction of League of Legends Competitive Matches Using Recurrent Neural Networks. Proceedings of SBGames, Computing Track - Short Papers, 639-642. Proceedings PDF (https://www.sbgames.org/sbgames2018/files/papers/ComputacaoShort/188226.pdf).

추가 구현 참고자료

• Riot Games, League of Legends 개발자 문서 (https://developer.riotgames.com/docs/lol)

• scikit-learn, Neural network models (supervised) (https://scikit-learn.org/stable/modules/neural_networks_supervised.html)

• scikit-learn, MLPClassifier (https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html)

• 로컬 실측 근거: docs/DEEP_LEARNING_RESULTS.md, docs/TEMPORAL_MODELS.md, docs/QA_FEATURE_EXTENSION_2026-10-30.md, docs/SUBMISSION_REQUIREMENTS.md

• Google, YouTube Data API search.list 및 videos.list (https://developers.google.com/youtube/data/reference/rest/v3/search/list, https://developers.google.com/youtube/data/reference/rest/v3/videos/list)

• OP.GG, 공식 MCP 서버 문서 (https://github.com/opgginc/opgg-mcp)

■ 부록 핵심 원천코드

다음 부록은 최초 MLP 검증본의 초기 특성 추출·모형 검증·추론과 학습 함수에서 발췌한 실제 원천코드를 당시 파일 해시와 함께 보존한다. 0.2.0 에서 추가하거나 재구성한 시간별 모형·입력 민감도·프로필·외부 연동·검사와 실행 스크립트는 현재 전체 소스 패키지에서 확인할 수 있다. 안전한 저장 모형 JSON 은 소스와 함께 별도로 제공하되 키·원시 경기 자료·캐시는 제외한다. 정확한 104 경기 재학습에는 비공개 원시 스냅샷이 필요하다.

backend/app/ml.py

최초 MLP 검증본 파일 SHA-256:

8360e82c41626f6c3907b08844c84c5cfeeac8189c43f96e37123503b4749ca6

_number

```
def _number(value: Any) -> float:
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise ValueError("Expected a finite number")
    try:
        result = float(value)
    except OverflowError as error:
        raise ValueError("Expected a finite number") from error
    if not math.isfinite(result):
        raise ValueError("Expected a finite number")
    return result
```

extract_features

```
def extract_features(match: dict, timeline: dict) -> dict[str, float] | None:
    """Blue-minus-red features; None for incomplete/ineligible snapshots.

    Never reads final outcomes or final player stats. Uses static team metadata
    and observations at or before minute 10. Rejects frames older than minute 9.
    Events are cut off at the selected frame's timestamp.
    """
    try:
        info = match["info"]
        if info.get("mapId") != 11 or info.get("queueId") not in (420, 440):
            return None
        if _number(info["gameDuration"]) <= 600:
```

```
    return None
participants = info["participants"]
if len(participants) != 10:
    return None
teams: dict[int, int] = {}
for participant in participants:
    pid, team = participant["participantId"], participant["teamId"]
    if type(pid) is not int or pid <= 0 or type(team) is not int:
        return None
    if pid in teams or team not in (100, 200):
        return None
    teams[pid] = team
if list(teams.values()).count(100) != 5 or list(teams.values()).count(200) !=
     5:
    return None
match_id = match.get("metadata", {}).get("matchId")
if not match_id or timeline.get("metadata", {}).get("matchId") != match_id:
    return None
frames = timeline["info"]["frames"]
eligible = [frame for frame in frames
            if MIN_SNAPSHOT_MS <= _number(frame["timestamp"]) <= SNAPSHOT_MS]
if not eligible:
    return None
frame = max(eligible, key=lambda item: item["timestamp"])
cutoff = frame["timestamp"]
pframes = frame["participantFrames"]
if set(pframes) != {str(pid) for pid in teams}:
    return None
result = dict.fromkeys(FEATURE_NAMES, 0.0)
for pid, team in teams.items():
    participant_frame = pframes[str(pid)]
    if participant_frame.get("participantId", pid) != pid:
        return None
    values = [_number(participant_frame[key]) for key in
              ("totalGold", "xp", "minionsKilled", "jungleMinionsKilled")]
    if any(value < 0 for value in values):
        return None
    sign = 1 if team == 100 else -1
    result["gold_diff"] += sign * values[0]
    result["xp_diff"] += sign * values[1]
    result["cs_diff"] += sign * (values[2] + values[3])
for event_frame in frames:
    if _number(event_frame["timestamp"]) > cutoff:
        continue
    for event in event_frame.get("events", []):
        if "timestamp" not in event or not 0 <= _number(event["timestamp"])
              <= cutoff:
            continue
        kind = event.get("type")
        credited_team, feature = None, None
        if kind == "CHAMPION_KILL":
            credited_team = teams.get(event.get("killerId"))
            feature = "kills_diff"
        elif kind == "BUILDING_KILL" and event.get("buildingType") ==
              "TOWER_BUILDING":
            # Riot's teamId is the destroyed tower's owner.
            owner = event.get("teamId")
            credited_team = 300 - owner if owner in (100, 200) else None
            feature = "towers_diff"
        elif kind == "ELITE_MONSTER_KILL" and event.get("monsterType") ==
              "DRAGON":
            credited_team = event.get("killerTeamId") or
                  teams.get(event.get("killerId"))
```

```
                    feature = "dragons_diff"
                if feature and credited_team in (100, 200):
                    result[feature] += 1 if credited_team == 100 else -1
        return result
    except (KeyError, TypeError, ValueError, OverflowError, AttributeError):
        return None
```

_read_model

```
def _read_model() -> dict | None:
    path = _model_path()
    if not path.is_file():
        return None
    if path.stat().st_size > 2_000_000:
        raise ValueError("Model artifact exceeds size limit")
    model = json.loads(path.read_text(encoding="utf-8"))
    algorithm = model.get("algorithm")
    if (model.get("schema_version"), algorithm) not in ((1, "logistic_regression"),
         (2, "mlp")):
        raise ValueError("Unsupported model format")
    if model.get("feature_names") != list(FEATURE_NAMES) or model.get("trained_on")
         not in ("riot", "synthetic"):
        raise ValueError("Invalid model provenance or features")
    for field in (model["scaler"]["mean"], model["scaler"]["scale"]):
        if len(field) != len(FEATURE_NAMES):
            raise ValueError("Invalid model dimensions")
        for value in field:
            _number(value)
    if any(value <= 0 for value in model["scaler"]["scale"]):
        raise ValueError("Invalid scaler")
    if algorithm == "logistic_regression":
        if len(model["coefficients"]) != len(FEATURE_NAMES):
            raise ValueError("Invalid logistic dimensions")
        for value in model["coefficients"]:
            _number(value)
        _number(model["intercept"])
    else:
        if (model.get("hidden_layers") != [16, 8, 4] or model.get("activation") !=
             "relu"
                or model.get("output_activation") != "logistic" or
                      model.get("parameter_count") != 289):
            raise ValueError("Unsupported neural network architecture")
        layers = model["layers"]
        if not isinstance(layers, list) or len(layers) != 4:
            raise ValueError("Invalid neural network layers")
        previous = len(FEATURE_NAMES)
        for layer, width in zip(layers, (16, 8, 4, 1)):
            weights, biases = layer["weights"], layer["biases"]
            if len(weights) != previous or len(biases) != width:
                raise ValueError("Invalid neural network dimensions")
            for row in weights:
                if len(row) != width:
                    raise ValueError("Invalid neural network dimensions")
                for value in row:
                    _number(value)
            for value in biases:
                _number(value)
            previous = width
    return model
```

predict_snapshot

```
def predict_snapshot(features: dict | None, team_id: int = 100, source: str = "riot")
    -> dict:
    info = get_model_info()
    response = {"probability": None, "method": "unavailable", "trained_on":
         info["trained_on"],
                "notice": info["notice"], "snapshot_minute": 10, "features": features
                     or {}}
    if not features:
        response["notice"] = "No complete eligible minute-10 snapshot is available
             for this match."
        return response
    if team_id not in (100, 200) or source not in ("riot", "synthetic"):
        response["notice"] = "Unsupported team or data provenance."
        return response
    if not info["available"]:
        return response
    if info["trained_on"] != source:
        response["notice"] = "Model and match data sources differ; this model cannot
             estimate this match."
        return response
    try:
        model = _read_model()
        if model is None or model["trained_on"] != source:
            return response
        values = [_number((_number(features[name]) - mean) / scale)
                  for name, mean, scale in zip(FEATURE_NAMES,
                        model["scaler"]["mean"], model["scaler"]["scale"])]
        if model["algorithm"] == "logistic_regression":
            z = _number(model["intercept"] + sum(coefficient * value
                                                 for value, coefficient in zip(values,
                                                         model["coefficients"])))
        else:
            for index, layer in enumerate(model["layers"]):
                values = [_number(bias + sum(value * row[column] for value, row in
                      zip(values, layer["weights"])))
                          for column, bias in enumerate(layer["biases"])]
                if index < len(model["layers"]) - 1:
                    values = [max(0.0, value) for value in values]
            z = values[0]
        probability_blue = 1 / (1 + math.exp(-max(-60.0, min(60.0, z))))
        response.update(probability=probability_blue if team_id == 100 else 1 -
             probability_blue,
                        method=model["algorithm"])
    except (OSError, ValueError, KeyError, TypeError, OverflowError, AttributeError):
        response["notice"] = "Snapshot or model is invalid; prediction is
             unavailable."
    return response
```

backend/scripts/train_model.py

최초 MLP 검증본 파일 SHA-256:

e5dac87b9fae0a77cab494cfb77c9df153fddc1393da98af6035b28e38dab6fa

train

```
def train(input_path: Path, output_path: Path, min_matches: int = 100, algorithm: str
    = "logistic_regression") -> dict:
    import numpy as np
    import sklearn
    from sklearn.linear_model import LogisticRegression
    from sklearn.neural_network import MLPClassifier
    from sklearn.exceptions import ConvergenceWarning
    from sklearn.metrics import accuracy_score, brier_score_loss, roc_auc_score
    from sklearn.preprocessing import StandardScaler

    if algorithm not in ("logistic_regression", "mlp"):
        raise ValueError("Unsupported training algorithm")
    if min_matches < 100:
        raise ValueError("At least 100 eligible unique matches are required")
    rows, seen, duplicates, skipped = [], set(), 0, 0
    input_bytes = input_path.read_bytes()
    for line_number, line in enumerate(input_bytes.decode("utf-8-sig").splitlines(),
         1):
        if not line.strip():
            continue
        try:
            record = json.loads(line)
            source = record["source"]
            if source not in ("riot", "synthetic"):
                raise ValueError("source must be riot or synthetic")
            match, timeline = record["match"], record["timeline"]
            features = extract_features(match, timeline)
            if features is None:
                skipped += 1
                continue
            info = match["info"]
            match_id = match["metadata"]["matchId"]
            timestamp = int(info.get("gameStartTimestamp", info.get("gameCreation",
                 0)))
            if timestamp <= 0:
                skipped += 1
                continue
            team_results = {team["teamId"]: team["win"] for team in info["teams"]}
            if set(team_results) != {100, 200} or any(type(value) is not bool for
                 value in team_results.values()):
                skipped += 1
                continue
            if team_results[100] == team_results[200]:
                skipped += 1
                continue
            if match_id in seen:
                duplicates += 1
                continue
            seen.add(match_id)
            rows.append({"match_id": match_id, "timestamp": timestamp, "source":
                 source,
                        "features": [features[name] for name in FEATURE_NAMES],
                        "label": int(team_results[100]), "queue": info["queueId"],
                        "patch": str(info.get("gameVersion", "unknown"))})
        except (KeyError, ValueError, TypeError) as exc:
            raise ValueError(f"Invalid input record on line {line_number}:
                 {type(exc).__name__}") from exc
    if len(rows) < min_matches:
        raise ValueError(f"Need at least {min_matches} eligible unique matches;
             received {len(rows)}")
```

```
sources = {row["source"] for row in rows}
if len(sources) != 1:
    raise ValueError("Do not mix real and synthetic data in a model or
         evaluation")
rows.sort(key=lambda row: (row["timestamp"], row["match_id"]))
split_index = int(len(rows) * 0.8)
boundary = rows[split_index]["timestamp"]
train_rows = [row for row in rows if row["timestamp"] < boundary]
test_rows = [row for row in rows if row["timestamp"] >= boundary]
if len(train_rows) < 60 or len(test_rows) < 20:
    raise ValueError("Need at least 60 earlier training and 20 later holdout
         matches")
y_train = np.array([row["label"] for row in train_rows])
y_test = np.array([row["label"] for row in test_rows])
if len(set(y_train)) != 2:
    raise ValueError("Training partition must contain both outcomes")
x_train = np.array([row["features"] for row in train_rows], dtype=float)
x_test = np.array([row["features"] for row in test_rows], dtype=float)
scaler = StandardScaler().fit(x_train)
scaled_train, scaled_test = scaler.transform(x_train), scaler.transform(x_test)
logistic = LogisticRegression(C=1.0, max_iter=2000,
     random_state=42).fit(scaled_train, y_train)
classifier, training_warnings = logistic, []
if algorithm == "mlp":
    # Fixed before this comparison; do not tune against the already viewed
         holdout.
    classifier = MLPClassifier(hidden_layer_sizes=(16, 8, 4), activation="relu",
         solver="lbfgs",
                               alpha=1.0, max_iter=2000, max_fun=20000, tol=1e-6,
                               random_state=42, early_stopping=False)
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always", ConvergenceWarning)
        classifier.fit(scaled_train, y_train)
    training_warnings = [str(item.message) for item in caught]
probabilities = classifier.predict_proba(scaled_test)[:, 1]

def measure(values, labels=y_test):
    return {"accuracy": float(accuracy_score(labels, values >= 0.5)),
            "brier_score": float(brier_score_loss(labels, values)),
            "roc_auc": float(roc_auc_score(labels, values)) if len(set(labels))
                  == 2 else None}

prior = float(y_train.mean())
baseline = np.full(len(y_test), prior)
source = next(iter(sources))
report = {
    "data_source": source,
    "synthetic_demo_only": source == "synthetic",
    "interpretation": ("Fictional generator evaluation only; NOT evidence of real
         League of Legends performance."
                       if source == "synthetic" else "Chronological holdout
                              estimate on this collected sample; no claim of
                              universal performance."),
    "input_sha256": hashlib.sha256(input_bytes).hexdigest(),
    "eligible_unique_matches": len(rows), "duplicates_removed": duplicates,
    "ineligible_records_skipped": skipped,
    "split": "chronological 80/20 target; equal boundary timestamps remain
         together in holdout",
    "scaler_fit": "training partition only", "selection": "one blue-side row per
         unique match",
    "train": _partition_report(train_rows), "test": _partition_report(test_rows),
    "metrics": measure(probabilities),
    "baseline": {"method": "constant training-set blue-win prevalence",
         "probability": prior,
```

```
                "accuracy": float(accuracy_score(y_test, baseline >= 0.5)),
                "brier_score": float(brier_score_loss(y_test, baseline)),
                "roc_auc": 0.5 if len(set(y_test)) == 2 else None},
    "limitations": ["One temporal holdout; no independent external validation or
         calibration study.",
                    "No hyperparameter tuning; patch drift and player-selection
                          bias may affect results.",
                    "Retrospective minute-10 analysis only; no live coaching or
                          causal claims.",
                    "Synthetic provenance is declarative; verify collection
                          provenance before real-data claims."],
    "libraries": {"numpy": np.__version__, "scikit_learn": sklearn.__version__},
}
if algorithm == "mlp":
    report["training_metrics"] =
         measure(classifier.predict_proba(scaled_train)[:, 1], y_train)
    report["logistic_baseline"] = {"method": "Logistic regression", "metrics":
         measure(logistic.predict_proba(scaled_test)[:, 1])}
    report["experiment_notice"] = ("The same holdout was previously viewed for
         the logistic baseline. This is an exploratory comparison, not fresh
         independent validation."
                                   if source == "riot" else "Synthetic generator
                                          comparison only; no empirical game-
                                          performance claim.")
    report["training"] = {"hidden_layers": [16, 8, 4], "activation": "relu",
         "output_activation": "logistic",
                          "solver": "lbfgs", "alpha": 1.0, "random_state": 42,
                                 "max_iter": 2000,
                          "max_fun": 20000, "tol": 1e-6, "early_stopping": False,
                          "iterations": int(classifier.n_iter_), "training_loss":
                                 float(classifier.loss_),
                          "warnings": training_warnings, "converged": not
                                 training_warnings}
    report["limitations"].append("Deep-network capacity is large relative to this
         small training set; overfitting remains possible.")
model = {"schema_version": 2 if algorithm == "mlp" else 1, "algorithm":
     algorithm, "trained_on": source,
         "created_at": datetime.now(timezone.utc).isoformat(), "feature_names":
              list(FEATURE_NAMES),
         "scaler": {"mean": scaler.mean_.tolist(), "scale":
              scaler.scale_.tolist()},
         "report": report}
if algorithm == "mlp":
    model.update(hidden_layers=[16, 8, 4], activation="relu",
         output_activation="logistic",
                 layers=[{"weights": weights.tolist(), "biases": biases.tolist()}
                         for weights, biases in zip(classifier.coefs_,
                                classifier.intercepts_)],
                 parameter_count=int(sum(weights.size + biases.size
                                         for weights, biases in
                                                 zip(classifier.coefs_,
                                                 classifier.intercepts_))))
else:
    model.update(coefficients=classifier.coef_[0].tolist(),
         intercept=float(classifier.intercept_[0]))
output_path.parent.mkdir(parents=True, exist_ok=True)
output_path.write_text(json.dumps(model, indent=2, allow_nan=False) + "\n",
     encoding="utf-8")
output_path.with_name(output_path.stem + "_evaluation.json").write_text(
    json.dumps(report, indent=2, allow_nan=False) + "\n", encoding="utf-8")
return model
```